Rapport Gestion d'une ligne de commande

1 Introduction

Une commande associe un client à plusieurs produits, et chaque produit vendu doit diminuer le stock. Le sujet de l'examen demande de gérer cette situation à partir d'un modèle comportant les clients, les commandes, les lignes de commande, les produits, les régions et les départements. Il faut enregistrer les produits d'une commande, afficher des listes, modifier un produit, retirer un produit d'une commande, puis fournir un tableau de bord statistique justifié.

La difficulté réelle n'est pas l'affichage mais la cohérence : que se passe-t-il si deux vendeurs commandent en même temps le dernier article, ou si une saisie échoue à mi-parcours ? Ce rapport montre comment l'application livrée répond à cette question, et ce qu'elle ne couvre pas encore.

2 Démarche et choix de conception

J'ai écarté tout framework au profit de PHP et de PDO, pour rester proche du sujet et pouvoir expliquer chaque ligne en soutenance. L'application s'installe sous XAMPP en copiant un dossier, et elle ne dépend d'aucun service externe : graphiques en SVG, icônes intégrées, feuille de style unique. Elle fonctionne donc sans Internet.

Chaque page a une fonction : le tableau de bord (index.php), le détail et la saisie d'une commande (commande.php), les trois listes, et la fiche d'un produit (produit.php). Le fichier inc/config.php regroupe ce que toutes les pages partagent : connexion, session, protection CSRF, échappement HTML, téléversement de photo et journal des requêtes.

La base est nommée tp_commande_2026 et ses tables portent le préfixe cmd_, afin d'éviter toute confusion avec une base précédente. Les colonnes gardent les noms du sujet (idCmd, qteCmd, prixUProd).

3 Réalisation

3.1 Le modèle de données

Six tables InnoDB reliées par des clés étrangères : un département contient des régions, une région contient des clients, un client passe des commandes. La table cmd_lignecommande a pour clé primaire le couple (idCmd, idProd) : un produit ne peut apparaître qu'une fois dans une commande. Une vue, cmd_v_ligne_detail, joint commande, ligne et produit et calcule le montant de chaque ligne ; toutes les statistiques s'appuient sur elle.

3.2 Enregistrer les produits d'une commande

C'est le cœur du système. Le formulaire accepte plusieurs lignes à la fois. Le serveur regroupe d'abord les doublons (même produit saisi deux fois, quantités additionnées), trie les produits par identifiant, puis exécute tout dans une transaction. L'extrait ci-dessous est simplifié par rapport au fichier réel :

```
$pdo->beginTransaction();
foreach ($lignes as $idp => $qte) {            // $lignes trié par ksort()
  SELECT designProd, qteStock FROM cmd_produit
   WHERE idProd = ? FOR UPDATE;                 // verrou sur le produit
  if (stock < $qte)  -> exception, message clair
  INSERT INTO cmd_lignecommande (idCmd, idProd, qteCmd)
   VALUES (?, ?, ?)
   ON DUPLICATE KEY UPDATE qteCmd = qteCmd + VALUES(qteCmd);
  UPDATE cmd_produit SET qteStock = qteStock - ? WHERE idProd = ?;
}
$pdo->commit();                                // sinon rollBack()
```

Trois mécanismes se combinent. La transaction garantit que la ligne et le stock changent ensemble ou pas du tout. Le verrou FOR UPDATE force un second vendeur à attendre la fin du premier, donc à voir le stock déjà diminué. Le tri préalable verrouille les produits toujours dans le même ordre, ce qui empêche deux transactions de s'attendre mutuellement. Si le stock est insuffisant pour un seul produit,

toute la saisie est refusée et le message indique la quantité disponible.

3.3 Retirer un produit, modifier un produit, lister

Le retrait d'une ligne suit la même logique en sens inverse : la ligne est verrouillée, supprimée, et sa quantité est rendue au stock dans la même transaction, après confirmation et uniquement par POST. La fiche d'un produit permet de modifier désignation, stock, prix et photo ; la photo est contrôlée d'après son contenu réel (JPG, PNG ou WebP, 2 Mo au plus) et enregistrée sous un nom aléatoire. Les listes affichent des vignettes (la photo, ou l'initiale sur fond coloré) et les montants en FCFA.

3.4 Le tableau de bord

Chaque indicateur répond à une question de gestion précise :

Question posée	Indicateur	Ce qu'on en tire
Les ventes vont-elles bien ?	CA, commandes, panier moyen, unités, clients actifs	État global en un coup d'œil
Quand vend-on ?	CA par mois (histogramme)	Tendance, saisonnalité
De quoi dépend-on ?	Top 5 produits, part du CA	Dépendance à quelques produits
Où vend-on ?	CA par département	Géographie des ventes
Qui fidéliser ?	Top 5 clients	Clients stratégiques
Que faut-il commander ?	Produits sous le seuil (10)	Action de réapprovisionnement

Un filtre de dates s'applique à tous les calculs, qui sont faits en SQL et non en PHP : le serveur web ne reçoit que des résultats agrégés.

4 Sécurité

Les requêtes sont préparées (PDO sans émulation), toutes les sorties passent par htmlspecialchars, et chaque formulaire porte un jeton CSRF vérifié avec hash_equals. Aucune suppression ne se fait par simple lien. Ces précautions sont modestes à écrire et couvrent les trois attaques les plus courantes sur ce genre d'application : injection SQL, XSS et requêtes forgées.

5 Analyse de performance

La performance repose sur trois décisions. D'abord la vue : la jointure à trois tables est écrite une fois, et les requêtes de statistiques restent courtes. Ensuite les index : un sur la date de commande (et un sur date et client) pour le filtre par période, d'autres sur les clés étrangères pour les jointures. Enfin les agrégations (SUM, COUNT, GROUP BY, LIMIT) sont déléguées à MySQL.

L'application se mesure elle-même : chaque page affiche son temps de génération et le nombre de requêtes, et le tableau de bord détaille la durée de chacune et propose un EXPLAIN dont la colonne key indique l'index utilisé. Ce rapport ne chiffre aucun temps, car je n'ai pas exécuté l'application : les mesures sont à relever à l'écran lors des tests.

6 Analyse critique

Points forts. Stock toujours cohérent, même en accès simultané ; sécurité de base complète ; tableau de bord justifié question par question ; installation en quelques minutes.

- **Prix non historisé :** le modèle ne conserve pas le prix au moment de la vente, donc le CA recalculé suit le prix actuel. Un prix dans la ligne de commande résoudrait le problème.
- **Pas de pagination ni de comptes utilisateurs :** suffisant pour un volume d'examen, insuffisant en production.
- **Identifiants de base dans le code :** à sortir du dépôt (variables d'environnement) avant tout déploiement.

7 Conclusion

L'application répond à toutes les demandes du sujet et traite le vrai problème de fond, la cohérence du stock, par des moyens simples et défendables : transaction, verrou et ordre de verrouillage constant. La prochaine étape logique serait d'historiser les prix et d'ajouter l'authentification.

Mise en route : copier le dossier dans htdocs, importer sql/01_schema_et_donnees.sql (sauf si la base d'examen existe), puis sql/02_vue_et_index.sql, vérifier inc/config.php et ouvrir http://localhost/gestion_commande/.